

LLM Notes

2026.9.3

Day 13: Nsight Systems：性能分析与日志解读

Profiling and Benchmarking

主讲：Aaron

Fudan University

今日摘要

2026.7.15 使用 Nsight Systems 分析 Transformer 的前向传播、反向传播及 AdamW 更新。这一部分感觉非常陌生，很多的日志统计结果看起来很混乱，只能交给 ai 整合分析。

13.1 实验设置与分析目标

本文主要分析以下代表配置：

- 模型：large（大约是 0.7B 左右），36 层， $d_{\text{model}} = 1280$ ， $d_{\text{ff}} = 5120$ ，20 个 attention heads；
- batch size： $B = 4$ ；序列长度： $S = 256$ ；
- 数据类型：FP32；
- 分析模式：forward、forward_backward、train_step。

其中，完整训练步骤包含前向传播、loss 计算、反向传播和 AdamW 更新。基础实验不包含梯度裁剪；梯度裁剪单独进行补充实验。

13.2 如何阅读统计日志

13.2.1 报告文件与命令

.nsys-rep 是采集结果文件，可以在 Nsight Systems GUI 中打开。nsys stats 用于生成统计表，首次运行时可能生成配套的 .sqlite 文件。

```
nsys stats --report cuda_gpu_kern_sum report.nsys-rep
```

```
nsys stats --report nvtx_kern_sum report.nsys-rep
```

两种统计表来自同一份采集记录，区别在于分组方式：

- `cuda_gpu_kern_sum`：按 kernel 名称汇总，适合判断整个模型的耗时构成；
- `nvtx_kern_sum`：按 NVTX 范围及 kernel 名称汇总，适合判断 attention、softmax、update 等代码范围的开销。

13.2.2 重点列的含义

Total Time 同一种 kernel 的累计执行时间。定位累计开销最大的 kernel 时，主要看这一列。

Instances 该 kernel 的调用次数。

Avg / Med 单次调用耗时的平均值与中位数。

Min / Max / StdDev 单次调用耗时的分布与波动。

Time (%) 当前行累计时间占表中全部 kernel 累计时间之和的比例，不是 GPU 利用率。

Name CUDA kernel 名称。

对于 NVTX kernel 表，还需注意：

- `NVTX Inst` 是标记范围的出现次数；
- `Kern Inst` 是对应 kernel 的调用次数；
- `PushPop` 表示 NVTX 标记采用入栈、出栈的范围形式。

13.2.3 统计规则与常见误区

1. 计算占比时优先使用原始时间，不直接累加已四舍五入的百分比。
2. 相同 kernel 可能被多个模型模块使用，不能仅凭名称将其全部归到某一个模块。

常见名称线索包括：

- `gemm / sgemm`：矩阵乘法相关；
- `elementwise / vectorized_elementwise`：逐元素操作；
- `reduce`：求和、最大值、均值等归约；
- `exp / sigmoid / sqrt / rsqrt`：对应数学运算；

- `copy / gather / scatter`: 复制或索引相关操作。

13.2.4 避免终端换行混乱

可以将统计表写入文本文件，再用编辑器阅读：

```
nsys stats --report cuda_gpu_kern_sum report.nsys-rep \  
> kernel_summary.txt
```

也可以使用横向滚动：

```
nsys stats --report cuda_gpu_kern_sum report.nsys-rep \  
| less -S
```

其中 `less -S` 避免长行自动折行，按 `q` 退出。若报告本身已经用省略号截断名称，横向滚动不能恢复被截断的内容。

13.3 如何阅读图形化时间线

13.3.1 CPU 与 GPU 轨道

打开 `.nsys-rep` 后，重点关注以下三类轨道：

Threads → **python3** → **NVTX** CPU 侧代码块范围，观察函数调用及等待时间。

Threads → **CUDA API** CPU 发起的 CUDA 调用，观察 kernel 提交和同步等待。

CUDA HW → **Kernel** GPU 实际执行的 kernel，观察计算、空隙及执行关系。

分析

只看 CPU 轨道，可能把等待 GPU 的时间误认为当前函数的计算时间；只看 GPU 轨道，则不容易解释为什么任务迟迟没有被提交。

例如 CPU 轨道中出现较长的 `cudaStreamSynchronize`，同时 GPU 仍有活动，说明 CPU 正在等待 GPU 工作完成。

13.3.2 Backward 的跨线程问题

`loss.backward()` 可以由 Python 主线程调用，但实际 CUDA 工作可能由 `pt_autograd` 工作线程发起。基于线程的 NVTX 关联不一定将工作线程的 kernel 归到主线程的 `backward` 标签下。

备注

GPU 投影中缺少 **backward** 标签，不等于没有执行反向传播，也不直接说明模型代码有错。应结合 autograd 工作线程的 CUDA API 和 GPU kernel 轨道判断。

13.4 cs336 五问：代表配置结果

13.4.1 (a) Forward 总耗时与 Python 计时

对 large 模型、序列长度 256，Nsight 测得包含结束同步的 CPU NVTX forward 平均耗时为 64.083 ms，与同次运行的 Python 计时 64.079 ms 几乎一致。不开启 profiler 时，Python 计时为 63.047 ms，开启采集后增加约 1.64%，表明本次采集对运行时间有小幅影响。

13.4.2 (b) 累计 GPU 时间最多的 kernel

Forward 中累计耗时最高的是 `tn...tilesize64x64x8` GEMM kernel，累计约 **25.18 ms**，调用 **72 次**，占全部 kernel 累计时间的 **41.4%**。加入 backward 后，第一名变为 `nn...tilesize256x128x8` GEMM kernel，在整个前向与反向范围内调用 **169 次**，累计约 **48.13 ms**，占比 **20.3%**。

13.4.3 (c) 非矩阵乘法开销

Forward 中，逐元素乘除、加法、归约及数据复制也占据不可忽略的运行时间，例如一种逐元素乘法 kernel 累计耗时约 **2.12 ms**。这些操作虽计算量较小，仍涉及 kernel 启动及数据读写，因此不能仅凭 FLOPs 判断其耗时贡献。

13.4.4 (d) 完整 AdamW 步骤的耗时构成

GEMM 相关 kernel 的累计时间占比从 forward 的 **82.8%** 降到完整训练步骤的 **71.7%**，其他 kernel 占比从 17.2% 升至 28.3%。这与反向传播和 AdamW 引入更多逐元素运算及归约相符，但不能将全局表中的所有新增操作都归属于 optimizer。

指标	Forward	Train step
全部 kernel 累计时间 (ms)	60.751	415.084
GEMM 相关累计时间 (ms)	50.327	297.660
GEMM 占比	82.8%	71.7%

13.4.5 (e) Attention: softmax 与矩阵乘法

36 层 softmax 的 GPU kernel 累计时间为 **2.554 ms**，QK 与 AV 操作范围合计为 **2.318 ms**，softmax 约为两者之和的 **1.10 倍**。虽然 softmax 的计算量更小，当前实现的多次中间结果读写、归约及 kernel 启动，使耗时没有随 FLOPs 同比例降低。

操作范围	累计时间 (ms)	平均每层 (μ s)
QK	1.158	32.16
Softmax	2.554	70.95
AV	1.161	32.24

本模型每个头的维度为：

$$d_h = \frac{1280}{20} = 64.$$

按一次乘加计为两次浮点运算，单层 QK 与 AV 的计算量合计约为：

$$F_{\text{QK+AV}} \approx 4BHS^2d_h = 4 \times 4 \times 20 \times 256^2 \times 64 \approx 1.342 \text{ GFLOPs}.$$

Softmax 对 BHS^2 个分数进行归约与逐元素操作，其工作量为 $O(BHS^2)$ ，没有矩阵乘法中额外的 d_h 因子。指数与比较操作的 FLOPs 计数存在约定差异，因此不据此给出精确的计算量倍数。

备注

AV 范围包含约 0.290 ms 的辅助逐元素操作。若只统计 QK、AV 中明确的 GEMM kernel，两者合计为 2.029 ms，softmax 约为其 1.26 倍。这些结果与 softmax 存在较大的访存和归约开销相符。

13.5 补充实验：梯度裁剪

13.5.1 正确的执行位置

梯度裁剪不是 AdamW 自动包含的步骤，需要在计算梯度后、更新参数前显式调用：

```
loss.backward()

with nvtx.range("clip_gradient"):
    clip_gradient(model.parameters(), max_grad_norm)

optimizer.step()
```

13.5.2 测量结果

在独立的含裁剪实验中：

指标	结果
裁剪关联 kernel 调用次数	1,313
Kernel 累计时间	9.563 ms
GPU 投影跨度	16.163 ms
CPU NVTX 区间	45.304 ms

参考实现逐个参数张量计算平方和，再逐张量缩放。本次有 327 个带梯度的参数张量，主要产生平方、求和、累加和缩放四类重复操作。

分析

CPU 的裁剪区间中出现明显的 `cudaStreamSynchronize` 等待。因此，45.304 ms 不能全部解释为裁剪自身的计算开销，其中可能包含等待先前排队的 GPU 工作。